

Exercice : mise en place d'un A/B testing sur Microsoft Azure

Description Générale

En tant que responsable DevOps, vous êtes chargé de déployer deux versions d'une même application afin de réaliser un test A/B.

Le test A/B (ou A/B testing) est une technique de marketing qui consiste à proposer plusieurs variantes d'un même objet qui diffèrent selon un seul critère (par exemple, la couleur d'un emballage) afin de déterminer la version qui donne les meilleurs résultats auprès des consommateurs. C'est une technique particulièrement employée dans la communication en ligne où il est maintenant possible de tester auprès d'un échantillon de personnes plusieurs versions d'une même page web, d'une même application mobile, d'un même courriel ou d'une bannière publicitaire afin de choisir celle qui est la plus efficace et de l'utiliser à large échelle.

Les applications sont volontairement simples afin que vous puissiez vous concentrer sur les aspects DevOps du projet. Vous pourrez ensuite enrichir les applications si vous le souhaitez.

Lors de la réalisation de votre objectif vous devrez produire des **images Docker légères et optimisées**, ainsi que construire un **pipeline structuré et maintenable**. L'optimisation des étapes de build et de test est attendue.

Partie 1 : Conteneurisation des applications

Objectif

Conteneuriser deux applications, une développée via Spring Boot (java), la seconde développée avec Django (python).

Version A : Spring Boot

- **Récupération du code** : Décompresser les sources du dépôt Git à l'adresse suivante :
 - [application Spring-Boot](#)
- **Fonctionnalités** :
 - La route / affiche une page d'accueil
- **Spécificités techniques** :
 - Version minimum du JDK : 21
 - Build : `mvn compile`
 - Lancement : `mvn spring-boot:run`
 - Tests : `mvn test`
 - Port : `8080`

Version B : Django

- **Récupération du code** : Décompresser les sources du dépôt Git à l'adresse suivante :
 - [application Django](#)
- **Fonctionnalités** :
 - La route / affiche la même page d'accueil avec un CSS différent
- **Spécificités techniques** :
 - Version minimum de python : 3.8
 - Préparation :
 - `pip install -r requirements.txt`
 - `python manage.py makemigrations`
 - `python manage.py migrate`
 - Lancement : `python manage.py runserver 0.0.0.0:8000`
 - Tests : `python manage.py test`
 - Port : `8000`

Travail à réaliser

- Créer un `Dockerfile` pour chaque application.
- Créer un fichier `docker-compose.yml` permettant de lancer les deux applications ensemble.
- Déposer sur le dépôt distant du projet :
 - Le code source des deux applications
 - Les `Dockerfile` des applications

?

- Le fichier `docker-compose.yml`
- Tous fichiers jugés utiles (`.gitignore`, `.dockerignore`, `README.md`,...)
- Ajouter le tag `conteneurisation` sur le dépôt distant à la fin de cette partie.

Partie 2 : Mise en place du routing A/B avec Nginx

Objectif

Mettre en place un conteneur Nginx qui redirige les utilisateurs entre les versions A et B. Pour réaliser cette seconde partie, la documentation officielle vous conduit à l'utilisation de la [directive upstream](#).

Travail à réaliser

- Créer un Dockerfile Nginx personnalisé.
- Configurer Nginx pour que chaque requête entrante sur la route `/test` soit redirigée vers la version A (spring-boot) ou B (Django).
- Mettre à jour le fichier `docker-compose.yml` créé à l'étape précédente pour lancer le serveur Nginx en plus des applications.
- Ajouter le tag `nginx` sur le dépôt distant à la fin de cette partie.

Partie 3 : Mise en place du pipeline CI

Objectif

Créer un pipeline CI pour tester et compiler les applications.

Travail à réaliser

- Définir un pipeline qui exécute les étapes suivantes :
 - Compilation des applications
 - Exécution des tests unitaires
- Ajouter le tag `pipeline-ci` sur le dépôt distant à la fin de cette partie.

N'oubliez pas de bien structurer votre dépôt et de documenter le fonctionnement de votre pipeline. A vous de déterminer le nombre de jobs de votre pipeline.

Partie 4 : Création de l'infrastructure Azure avec Terraform

Objectif

Déployer l'infrastructure nécessaire sur Microsoft Azure pour héberger les deux application (Spring-boot et Django).

Travail à réaliser

- Imaginer les types et la quantité de ressources nécessaires (App Services, Container Registry,...).
- Créer le projet Terraform pour créer l'infrastructure imaginée.
- Déployer les deux applications sur cette infrastructure.
- Vérifier que les applications sont opérationnelles et accessibles via leur route `/`.
- Ajouter le tag `infrastructure` sur le dépôt distant à la fin de cette partie.

Remarque : Le script Terraform doit être versionné dans le dépôt Git, mais aucun job du pipeline ne doit automatiser cette étape. L'exécution du plan Terraform reste manuelle.

Partie 5 : Automatisation du déploiement via le pipeline

Objectif

Intégrer le déploiement des deux applications dans le pipeline CI/CD.

Travail à réaliser

- Ajouter un **job de déploiement** au pipeline CI/CD permettant de :
 - Construire les images Docker
 - Déployer les services sur l'infrastructure Azure créée avec Terraform
- Ajouter le tag `déploiement` sur le dépôt distant à la fin de cette partie.

Notes :

- Pour cette étape, vous aurez probablement besoin d'enregistrer le gitlab-runner en précisant le volume vers le socket Docker. Reportez-vous à la fin du TD sur GitLab CI pour plus de détails à ce sujet.
- Si vous devez vous connecter à votre ACR, la commande de connexion `docker login "$ACR_NAME.azurecr.io" -u "$ARM_CLIENT_ID" --password "$ARM_CLIENT_SECRET"` vous sera utile. Les différentes variables liées à votre compte Microsoft Azure devront être sauvegardées dans les variables votre projet.
- À la fin des parties 3 et 4, vous devez détruire l'infrastructure Azure créée afin de ne pas consommer inutilement vos crédits Azure. Vos scripts Terraform et vos pipelines doivent permettre de recréer l'infrastructure et de redéployer les applications facilement le jour de votre présentation. Assurez-vous que tout soit reproductible sans intervention manuelle complexe.

Partie 6 : Mise à jour du proxy

Mettre en place un nouveau conteneur Nginx qui redirige les utilisateurs entre les versions A et B déployées sur Microsoft Azure.

Travail à réaliser

- Créer un Dockerfile Nginx personnalisé.
- Configurer Nginx pour que chaque requête entrante sur la route `/test` soit redirigée vers la version A (spring-boot) ou B (Django) déployée sur Microsoft Azure.
- Ajouter le tag `nginx-azure` sur le dépôt distant à la fin de cette partie.

Notes : - Si vous souhaitez aller plus loin, vous pouvez essayer la ressource **Azure Front Door** qui permet de gérer des test A/B.

Modifié le: mercredi 15 avril 2026, 16:39